

GUÍA Y DOCUMENTACIÓN PASO A PASO: UNIDAD III (MEMORIA)

Este documento sigue el mismo formato y estilo utilizado en la **Unidad 1 y Unidad 2**. Contiene la explicación en lenguaje claro, el procedimiento paso a paso en la terminal de **Red Hat Enterprise Linux (RHEL)**, la interpretación de resultados y la preparación para la defensa individual.

1. El Problema: ¿De qué se trata?

Imaginate que la memoria RAM es como un estacionamiento dividido en 5 cocheras de diferentes tamaños (100, 500, 200, 300 y 600 KB). Van llegando autos (procesos) que necesitan estacionar en un solo bloque continuo:

- **Primer Ajuste (First-Fit):** El auto entra en la primera cochera donde quepa, sin importar si sobra mucho espacio. Esto parece rápido, pero arruina las cocheras grandes metiendo autos medianos.
- **Mejor Ajuste (Best-Fit):** El auto recorre todo el estacionamiento y elige la cochera que le quede más justa para no desperdiciar tanto lugar.

El nudo del problema: En First-Fit, cuando llega el auto grande \$P_4\$ (426 KB), no entra en ningún lugar y es **rechazado**, a pesar de que si sumamos todos los pedazos libres sobran **959 KB**. Eso es exactamente la **fragmentación externa**.

2. Paso a Paso en la Consola de Red Hat

Paso 1: Entrar a la carpeta de trabajo

Abrir la terminal de Red Hat y posicionarse en la carpeta de la materia:

```
Bash
cd ~/sistemas-operativos
git pull origin main
mkdir -p Unidad_3
cd Unidad_3
```

Paso 2: Crear el simulador en C (simulador_memoria.c)

Paso 3: Crear el script de ejecución (ejecutar_tpo.sh)

Paso 4: Dar permisos y ejecutar

```
Bash
chmod +x ejecutar_tpo.sh
./ejecutar_tpo.sh
```

3. ¿Qué significa la salida de la pantalla?

Al correr ./ejecutar_tpo.sh, la consola muestra exactamente esto:

A. En Primer Ajuste (First-Fit):

A. En Primer Ajuste (First-Fit):

- P_1 (212 KB) se mete en el Hueco 2 (de 500 KB). PNG
- P_2 (417 KB) entra en el Hueco 5 (de 600 KB). PNG
- P_3 (112 KB) entra en lo que sobró del Hueco 2. PNG
- El fallo con P_4 : Llega P_4 pidiendo 426 KB y el sistema dice:
P4 426 KB Ninguno RECHAZADO (Sin espacio contiguo)
PNG
- Diagnóstico: Abajo dice Memoria total libre no contigua: 959 KB . Hay 959 KB libres en total, pero P_4 no entra porque ningún pedacito individual llega a 426 KB. Eso es Fragmentación Externa. PNG + 4

B. En Mejor Ajuste (Best-Fit):

B. En Mejor Ajuste (Best-Fit):

- Busca el hueco más justo para cada uno: P_1 va al Hueco 4 (300 KB), P_2 va al Hueco 2 (500 KB), P_3 va al Hueco 3 (200 KB) y P_4 entra en el Hueco 5 (600 KB). PNG + 1
- Resultado: Los 4 procesos quedaron ASIGNADOS y la memoria libre sobrante bajó a 533 KB. PNG + 1

C. Comparación de tiempos (time):

C. Comparación de tiempos (time):

- real 0m0.001s / sys 0m0.001s en ambos casos. PNG
- Por qué First-Fit es más rápido: Tiene un break en el código en C: apenas encuentra un hueco donde entra, asigna y sale. Best-Fit tiene que recorrer siempre los 5 huecos hasta el final para comparar cuál es el más chico. PNG + 3

4. Subir la entrega a GitHub

Una vez que comprobaste que funciona, subís los archivos a tu repositorio:

5. Guía para la Exposición Oral (Defensa Individual)

Si te eligen para exponer, explicás estos tres puntos en orden:

1. El Problema (30 segundos):

"En nuestro servidor Red Hat simulamos un motor de base de datos que asigna memoria continua. Los usuarios se quejaban de que el sistema rechazaba programas diciendo que no había memoria, pero en los comandos de monitoreo se veían casi 1000 KB libres. Quisimos demostrar por qué pasa esto."

2. La Demostración (1 minuto):

Mostrás la pantalla y señalás:

"Con Primer Ajuste vemos que **P4** queda RECHAZADO y abajo sobran 959 KB. Con Mejor Ajuste los 4 procesos entran perfecto porque eligió los huecos a medida."

3. Las Preguntas Clave del Cuestionario (El remate teórico):
 - **¿Qué es la fragmentación externa? (Pregunta 32):** Es cuando hay suficiente memoria libre total en el sistema para satisfacer una solicitud, pero no se puede usar porque está partida en pedacitos dispersos que no son continuos.
 - **¿Cómo se soluciona en la teoría y en la práctica? (Preguntas 33 y 34):** En la teoría se soluciona con **compactación** (mover los programas para juntar los huecos, pero gasta mucha CPU). En los sistemas operativos modernos como Red Hat se soluciona con la **paginación**, dividiendo los programas en páginas y la memoria en marcos de 4 KB, permitiendo que un proceso esté guardado en partes separadas sin necesidad de ser continuo.